

CodeAtlas — Guía de exposición

15 minutos de explicación + 5 minutos de demo. Tiempos orientativos: ajusta en el ensayo.

Regla de oro: poca letra en pantalla; lo importante lo cuentas tú.

Diapositiva 1 — Portada

- Saluda y preséntate.
- Presenta CodeAtlas: una herramienta que convierte el código en un mapa visual e interactivo.
- Formato de hoy: 15 minutos de explicación + 5 minutos de demo en vivo.

Diapositiva 2 — El problema — entender una app

- Entender una aplicación nueva cuesta días leyendo código.
- Quien entra nuevo a un proyecto tarda mucho en ser productivo.

Diapositiva 3 — El problema — mantener los diagramas

- Ya existen formatos para dibujar apps; el problema no es dibujarlas, es mantenerlas.
- Cada diagrama se actualiza a mano y por separado, así que se desfasa en cuanto cambia el código.
- Además están dispersos en herramientas distintas (esquema de BD, casos de uso, flujos).
- CodeAtlas aporta un único origen de verdad: todos los diagramas salen de ahí, sincronizados y centralizados.

Diapositiva 4 — De dónde viene la idea

- La IA es muy buena traduciendo lenguaje humano a lenguaje de máquina y al revés.
- Puede analizar una app y pasarla a un formato concreto que una herramienta sabe dibujar.
- De ahí nace CodeAtlas: transforma la aplicación en un esquema visual (módulos, pantallas, BD, flujos).
- Caso estrella: que un trabajador nuevo entienda la app en minutos, no en días.

Diapositiva 5 — No solo para entenderla entera

- No solo sirve para ver la app completa de golpe.
- En el día a día: antes de tocar código, ves cómo funcionan los archivos que vas a modificar y cómo se conectan.
- Así sigues el mismo patrón del proyecto y no metes código que rompe la estructura.

Diapositiva 6 — La visión a futuro

- Hoy ya programamos con agentes de IA (p. ej. Claude Code): defines la idea y le pides que la implemente.
- La clave: un skill que haga que cada vez que el agente escribe código, genere y actualice la documentación automáticamente.
- Así la documentación —y el diagrama— quedan siempre al día en tiempo real, sin esfuerzo extra.

Diapositiva 7 — ¿Qué hace CodeAtlas?

- Subes una carpeta de archivos .md, CodeAtlas los interpreta y genera un diagrama interactivo.
- En el diagrama ves módulos, pantallas, base de datos y flujos, y cómo se conectan entre sí.
- No es estático: puedes mover nodos, filtrar y hacer clic para ver detalles.

Diapositiva 8 — Lenguajes y tecnologías

- JavaScript de punta a punta (Vue delante, Node detrás): la razón es que son las que mejor conozco.
- Frontend: Vue 3 + Vite + Pinia + Vue Router + Vue Flow (el lienzo de nodos y conexiones).
- Backend: Node.js + Express (API REST), base de datos MySQL e integración de IA con Google Gemini.

Diapositiva 9 — Arquitectura modular

- Cada área funcional vive en su propia carpeta (organización por módulo, no por tipo de archivo).
- Backend, mismo patrón en cada módulo: routes → controller → service → repository.
- Frontend, mismo patrón: views → components → services → stores.
- Ventaja: fácil de entender, mantener y ampliar (una funcionalidad nueva es una carpeta más).
- Frontend y backend separados, comunicados por una API REST.

Diapositiva 10 — Base de datos

- Base de datos relacional en MySQL, con 6 tablas relacionadas.
- Tablas: users, user_settings, projects, diagrams, bot_sessions y bot_files.
- La tabla diagrams guarda el modelo y el layout del canvas en JSON.
- Seguridad: toda consulta filtra por user_id y los borrados son en cascada.

Diapositiva 11 — El formato .md que interpreta

- CodeAtlas no adivina: lee archivos .md con un formato muy concreto.
- Hay 7 tipos de archivo; cada archivo describe una pieza y se convierte en un nodo del diagrama.
- El frontmatter (metadatos y referencias por ID) genera las flechas del diagrama.
- Las secciones ## son el contenido que se ve al pulsar el nodo.

Diapositiva 12 — Anatomía de un archivo

- Ejemplo con un módulo de backend.
- Los campos con IDs (database, depends-on) le dicen al parser con qué conectar el nodo: esas son las flechas.
- Las secciones ## Purpose y ## Functions son lo que aparece en el panel de detalle del nodo.

Diapositiva 13 — El mismo formato, otras piezas

- Pasa rápido por los ejemplos: índice de módulos, pantalla, módulo de frontend y tabla de BD (DBML).

- Es el mismo formato aplicado a piezas distintas.

Diapositiva 14 — El flujo

- Es el tipo más especial: una secuencia de pasos que cruza las capas de la app.
- Cada paso lleva un prefijo [capa:módulo/archivo/función].
- El parser lo convierte en un recorrido que se ilumina paso a paso: pantalla → frontend → backend → base de datos.
- Permite ver qué pasa cuando el usuario hace algo, sin leer código.

Diapositiva 15 — El pipeline del parser

- Al subir los archivos, el backend ejecuta un pipeline lineal.
- Pasos: extraer → validar → construir el modelo → resolver referencias → calcular el layout.
- El resultado es un JSON que el frontend dibuja con Vue Flow.
- Se guardan el modelo y las posiciones, así al reabrir el diagrama queda tal cual lo dejaste.

Diapositiva 16 — El asistente de IA

- Le describes tu app en lenguaje natural y Gemini genera los .md con el formato correcto.
- Conversa, mantiene el historial por sesión y valida los archivos antes de guardarlos.
- Los descargas como un .zip listo para generar el diagrama.
- Eliges el modelo (Flash / Flash-lite) y, si se agota la cuota, te ofrece cambiar. La API key vive solo en el servidor.
- Cubre el requisito del ciclo de integrar un modelo de IA.

Diapositiva 17 — Seguridad

- Autenticación con JWT (token firmado que caduca).
- Contraseñas hashadas con bcrypt, nunca en texto plano.
- Aislamiento: toda consulta filtra por user_id, así nadie ve datos de otro.
- Entradas validadas y consultas parametrizadas para evitar inyección SQL.

Diapositiva 18 — Desplegada en producción

- App contenerizada con Docker y desplegada en Kubernetes.
- Servidor interno del instituto: el acceso es por HTTP (no hay HTTPS), con URL interna.
- Cumple el requisito de tener la app desplegada (sin FTP) y con frontend/backend separados.

Diapositiva 19 — Próximos pasos

- Analizar aplicaciones ya existentes directamente desde el código (application-diagram).
- Generar código alineado con el diagrama.
- Trabajo en equipo y diagramas compartidos.

Diapositiva 20 — Cierre / Demo

- Recap en una frase: CodeAtlas convierte la documentación de una app en un mapa visual, con una IA que genera ese formato por ti.
- Cierra: “Y ahora, en vez de seguir contándolo, os lo enseño funcionando” y pasa a la demo.

Demo en vivo (5 minutos)

- Login: entra rápido (ten las credenciales listas).
- Dashboard: enseña tus proyectos y los diagramas recientes.
- Asistente IA: describe una app sencilla, deja que genere los .md y descarga el zip.
- Generar diagrama: crea uno nuevo, sube los .md y muestra el progreso por fases.
- Canvas (lo más vistoso): mueve nodos, usa undo/redo, cambia a modo Flujos (recorrido paso a paso), haz clic en un nodo para ver el detalle y prueba los filtros.
- Cierre: vuelve al dashboard y di "Gracias".
- Plan B (si falla la red o la IA): salta directo al Canvas con un diagrama ya generado; es lo más vistoso y no depende de internet.